

CONTAINERIZE AND DEPLOY THE PYTHON SERVICE TO CLOUD RUN

Lab Manual

Four labs on two supplied Python services — Cloud Shell only, nothing to code.

Contents

How to use this manual.....	3
Lab 1 — Build the image twice and compare.....	4
Lab 2 — Push, deploy with sizing, timeout, cold start.....	9
Lab 3 — Configure, split, roll back, read the logs, tear down.....	13
Appendix — Environment, versions and the kit.....	16

How to use this manual

The slides teach the concepts and point to a lab; this manual has every step. The service (bookshelf-api) and both Dockerfiles are supplied complete in the kit — you read them, build them, push, deploy and observe. Nothing is edited.

Conventions

- Cloud Shell blocks (teal) are copy-pasteable. Run them one block at a time, in order; a few steps wait 30–45 seconds on purpose.
- Expected panels show the real output format. Sizes, hashes, IPs and revision names differ in your project; the shape is what matters.
- Console path lines give the equivalent clicks where one exists. Local Docker steps have no console equivalent.
- Each lab ends with a Check yourself box and a Troubleshooting table. Lab 4 ends with the teardown — run it.

Getting the kit into Cloud Shell

Upload gcp-cloudrun-labs-all.zip through the Cloud Shell menu (: → Upload) or drag it onto the terminal, then:

```
$ unzip -q gcp-cloudrun-labs-all.zip && cd gcp-cloudrun-labs && ls
README.md  bookshelf-api  bookshelf-web  labs.sh  teardown.sh
$ ls bookshelf-api
Dockerfile  Dockerfile.naive  README.md  app.py  gunicorn.conf.py  requirements.txt
```

labs.sh is the same command sheet as this manual, in one file — for pasting, never for running end to end.

Cost note

A service left with min-instances 1 costs roughly US\$5–10 a month idle; a few images in Artifact Registry cost cents; a Cloud Shell disk full of images costs nothing but breaks the next session. Lab 3's teardown removes all three.

Lab 1 — Build the image twice and compare

Estimated time	Difficulty	Environment	Kit
30 min	●●○	Cloud Shell, region asia-south1	gcp-cloudrun-labs-all.zip (bookshelf-api/ + labs.sh + teardown.sh)

OBJECTIVE

The naive image and the hardened image built side by side, four numbers compared (size, pip layer, uid, gcc), the hardened image running locally, and a clean SIGTERM stop timed and read.

PREREQUISITES

- Cloud Shell open on your sandbox project (Docker is pre-installed).
- Kit unzipped; you are inside gcp-cloudrun-labs/bookshelf-api.

Steps

Step 1 — Read the service and both Dockerfiles (no edits) and enable the APIs

CLOUD SHELL

```
cd bookshelf-api
cat app.py gunicorn.conf.py Dockerfile.naive Dockerfile .dockerignore
gcloud services enable run.googleapis.com cloudbuild.googleapis.com
artifactregistry.googleapis.com
```

The API enable is harmless if they are already on.

Step 2 — Build the naive image and look at it

python:3.13 is ~1 GB; the first pull takes 1–2 minutes.

CLOUD SHELL

```
docker build -f Dockerfile.naive -t bookshelf-api:naive .
docker images bookshelf-api
docker history bookshelf-api:naive --format 'table {{.Size}}\t{{.CreatedBy}}' | head -8
docker run --rm bookshelf-api:naive id -u
```

EXPECTED

```
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
bookshelf-api naive     7c1e9a2b4d3f   30 seconds ago 1.05GB
SIZE          CREATED BY
0B            CMD ["gunicorn" "app:app"]
12.4MB        RUN /bin/sh -c pip install flask gunicorn
9.1kB         COPY . /app
0
# ← uid 0: root
```

Step 3 — Build the hardened multi-stage image and compare the same numbers

CLOUD SHELL

```
docker build -t bookshelf-api:1.0 .
docker images bookshelf-api
docker history bookshelf-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -12
docker run --rm bookshelf-api:1.0 id -u
docker run --rm bookshelf-api:1.0 sh -c 'which gcc || echo no-gcc'
```

EXPECTED

```
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
bookshelf-api 1.0       e4a71c9d2b08   20 seconds ago 162MB
```

```
bookshelf-api  naive  7c1e9a2b4d3f  3 minutes ago  1.05GB
1000
no-gcc
```

	naive	1.0
size	~1.05 GB	~160 MB
pip layer	unpinned (pip install flask gunicorn)	Flask==3.1.3, gunicorn==26.2.0
uid	0 (root)	1000 (appuser)
gcc in the runtime image	yes	no
a code change rebuilds	the pip layer too (COPY . comes first)	the code layer only

Step 4 — Run the hardened image locally and call the contract endpoints

Console path: Cloud Shell → Web Preview (icon top right) → Preview on port 8080 shows the JSON in a browser tab.

CLOUD SHELL

```
docker run -d --name api -p 8080:8080 bookshelf-api:1.0
curl -s localhost:8080/healthz; echo
curl -s localhost:8080/whoami; echo
curl -s localhost:8080/books/2; echo
```

EXPECTED

```
{"status":"ok","uptime_s":4.2}
{"instance":"3a9f0c1e7b2d","uptime_s":6.9,"version":"v1"}
{"author":"Kleppmann","id":2,"title":"Designing Data-Intensive
Applications","year":2017}
```

Port busy? Use -p 8081:8080 and curl 8081.

Step 5 — The contract, measured — time a stop and read the shutdown line

docker stop sends SIGTERM, then SIGKILL after 10 s. An exec-form CMD with gunicorn as PID 1 stops in about a second and logs the on_exit line; a shell-form CMD would take the full 10 s and exit 137.

CLOUD SHELL

```
time docker stop api
docker logs api | tail -2
docker inspect api --format '{{.State.ExitCode}}'
docker rm api
```

EXPECTED

```
api
real    0m1.1s
[INFO] Shutting down: Master
bookshelf-api: SIGTERM received, in-flight requests finished, exiting cleanly
0
```

CHECK YOURSELF

- Two images listed: naive ~1 GB, 1.0 ~160 MB.
- id -u printed 1000 for 1.0 and 0 for naive.
- docker stop took about a second and the log ends with the clean-shutdown line; exit code 0.

Troubleshooting

Symptom	Cause	Fix
permission denied on docker.sock	Not in Cloud Shell	Use Cloud Shell; Docker is pre-installed there
No space left on device	Cloud Shell's 5 GB home is full of images	docker system prune -af, then rebuild
curl: connection refused on 8080	Container not up yet, or port in use	docker ps; docker logs api; or use -p 8081:8080
stop took ~10 s, exit 137	You ran the naive image (shell-form CMD)	Run bookshelf-api:1.0 — the hardened one

Challenge (fast finishers)

Change requirements.txt's Flask pin to Flask==3.1.2 in a scratch copy of the folder, rebuild, and read docker history: which layers rebuilt and which were cached? Then put the change in app.py instead and compare.

Summary

Seven times smaller, pinned, non-root, no compiler — and app.py did not change by a character. The stop timing is how you check the container contract before anything reaches Cloud Run.

Lab 2 — Push, deploy with sizing, timeout, cold start

Estimated time	Difficulty	Environment	Kit
30 min	...	Cloud Shell, region asia-south1	gcp-cloudrun-labs-all.zip (bookshelf-api/ + labs.sh + teardown.sh)

OBJECTIVE

The hardened image pushed by Cloud Build into Artifact Registry, deployed to Cloud Run with every sizing flag explicit, a 504 provoked and fixed, and a cold start measured and removed.

PREREQUISITES

- Lab 1 completed (the hardened image exists locally — not required for the push, Cloud Build rebuilds it).
- You are inside gcp-cloudrun-labs/bookshelf-api.

Steps

Step 1 — Create the repository and push with Cloud Build

gcloud builds submit zips the folder, runs the Dockerfile on a Cloud Build worker and pushes the image. 'already exists' on the repo is fine.

Console path: Artifact Registry → Create repository → bookshelf, Docker, asia-south1. Cloud Build → History shows the build. Artifact Registry → bookshelf → bookshelf-api → digest, tags, Vulnerabilities tab.

CLOUD SHELL

```
AR=asia-south1-docker.pkg.dev/$(gcloud config get-value project)/bookshelf
gcloud artifacts repositories create bookshelf --repository-format=docker --
location=asia-south1
gcloud builds submit --tag $AR/bookshelf-api:1.0
gcloud artifacts docker images list $AR --include-tags
```

EXPECTED

```
Created repository [bookshelf].
Step #0: Step 1/12 : FROM python:3.13-slim AS builder
Step #0: Step 6/12 : FROM python:3.13-slim
PUSH ... 1.0: digest: sha256:9e1d5c7a3b4f... size: 1786
ID      CREATE_TIME      DURATION  SOURCE      IMAGES
STATUS
a41c... 2026-09-07T10:12:40+00:00 58s      gs://.../source .../bookshelf/bookshelf-
api:1.0      SUCCESS
IMAGE                                     DIGEST                                     TAGS
CREATE_TIME
asia-south1-docker.pkg.dev/.../bookshelf/bookshelf-api  sha256:9e1d5c7a3b4f...  1.0
2026-09-07T10:13:31
```

About a minute. Answer y if asked to enable an API.

Step 2 — Deploy the image with explicit sizing

Console path: Cloud Run → Deploy container → Service → select the image from Artifact Registry → Container: CPU 1, Memory 512 MiB, Request timeout 30, Max concurrent requests 80; Autoscaling max 5; Allow unauthenticated → Create.

CLOUD SHELL

```
gcloud run deploy bookshelf-api --image $AR/bookshelf-api:1.0 --region asia-south1 \
--cpu 1 --memory 512Mi --concurrency 80 --max-instances 5 --timeout 30 --allow-
unauthenticated
URL=$(gcloud run services describe bookshelf-api --region asia-south1 --
format='value(status.url)')
curl -s $URL/healthz; echo; curl -s $URL/books/2; echo
```

EXPECTED

```
Service [bookshelf-api] revision [bookshelf-api-00001-x9k] has been deployed and is
serving 100 percent of traffic.
Service URL: https://bookshelf-api-482915730261.asia-south1.run.app
{"status":"ok","uptime_s":2.7}
{"author":"Kleppmann","id":2,"title":"Designing Data-Intensive
Applications","year":2017}
```

Step 3 — Watch instances — call /whoami ten times

CLOUD SHELL

```
for i in $(seq 10); do curl -s $URL/whoami; echo; done
```

EXPECTED

```
{"instance":"localhost","uptime_s":3.1,"version":"v1"}
{"instance":"localhost","uptime_s":3.2,"version":"v1"}
... # the same instance keeps answering: ten sequential requests never exceed
concurrency 80
```

The hostname reads 'localhost' on Cloud Run; uptime_s is the useful field.

Step 4 — Provoke the timeout, then raise it

A 45-second request against a 30-second timeout: Cloud Run cuts the response at exactly 30 s with a 504 while the handler keeps running. Raising the timeout creates a new revision.

Console path: Cloud Run → bookshelf-api → Edit & deploy new revision → Container → Request timeout 120 → Deploy.

CLOUD SHELL

```
curl -s -o /dev/null -w '%{http_code} in %{time_total}s\n' "$URL/slow?seconds=45"
gcloud run services update bookshelf-api --region asia-south1 --timeout 120
curl -s -o /dev/null -w '%{http_code} in %{time_total}s\n' "$URL/slow?seconds=45"
```

EXPECTED

```
504 in 30.021s
✓ Deploying... Done. Service [bookshelf-api] revision [bookshelf-api-00002-p4m] ...
serving 100 percent of traffic.
200 in 45.087s
```

Each curl really waits; run them once.

CHECK YOURSELF

- The image is in Artifact Registry with a digest.
- The first slow call returned 504 at ~30 s; the second returned 200 at ~45 s.
- Revisions tab now shows two revisions — the timeout change made the second.

Step 5 — Measure a cold start — a fresh revision forces a fresh instance

CLOUD SHELL

```
gcloud run services update bookshelf-api --region asia-south1 --update-env-vars
MARK=cold
curl -s -w '\ntotal %{time_total}s\n' $URL/whoami # first hit: cold
curl -s -w '\ntotal %{time_total}s\n' $URL/whoami # second hit: warm
```

EXPECTED

```
✓ Deploying... Done. revision [bookshelf-api-00003-c7n] ...
{"instance":"localhost","uptime_s":0.4,"version":"v1"}
total 1.317s
{"instance":"localhost","uptime_s":6.9,"version":"v1"}
total 0.064s
```

Your cold number will differ (0.6–2.5 s for this image); the warm number is what matters.

Step 6 — Remove it — keep one instance warm and boost start-up CPU

`min-instances` removes the cold start after idle (which you cannot wait 15 minutes to see in class); the first hit of any brand-new revision is still cold. The Metrics tab shows the instance count now flooring at 1.

Console path: Edit & deploy new revision → Container → Startup CPU boost; Autoscaling → Minimum instances 1. Then Metrics → Container instance count.

CLOUD SHELL

```
gcloud run services update bookshelf-api --region asia-south1 --min-instances 1 --cpu-boost
```

EXPECTED

```
✓ Deploying... Done. revision [bookshelf-api-00004-k2d] ...
```

Troubleshooting

Symptom	Cause	Fix
deploy: image not found	\$AR unset in this shell	Re-run the first line of step 1
builds submit: permission denied	cloudbuild API off	Step 1 of Lab 1 (enable), answer y
504 never appears	Timeout not 30 — a later deploy raised it	<code>gcloud run services update ... --timeout 30</code> , repeat
Both curls in step 5 are fast	The new revision's instance was already warm from the update	Repeat step 5 with <code>MARK=cold2</code>

Challenge (fast finishers)

Fire 200 requests in parallel and count distinct instances: `seq 200 | xargs -P 50 -I{} curl -s $URL/whoami | grep -o 'uptime_s':[0-9.]*' | sort | uniq -c`. Then set `--concurrency 8` and repeat — how many instances now?

Summary

Build once into the registry, deploy by reference with sizing you chose, and let the platform hold you to it: the 504 arrived at the number you typed, and the cold start is a number you measured before paying `min-instances` to remove it.

Lab 3 — Configure, split, roll back, read the logs

Estimated time	Difficulty	Environment	Kit
15 min	●●○	Cloud Shell, region asia-south1	gcp-cloudrun-labs-all.zip (bookshelf-api/ + labs.sh + teardown.sh)

OBJECTIVE

A new revision from environment variables alone, a 50/50 split, a rollback, the logs including the clean-shutdown line, and the revision history.

PREREQUISITES

- Lab 2 completed: bookshelf-api is running with several revisions; \$URL is set (re-run the describe line from Lab 2 step 2 if the shell is new).

Steps

Step 1 — Change configuration only — a new revision

Console path: Cloud Run → bookshelf-api → Edit & deploy new revision → Variables & secrets → GREETING = Hello from v2, APP_VERSION = v2 → Deploy.

CLOUD SHELL

```
gcloud run services update bookshelf-api --region asia-south1 --update-env-vars
GREETING='Hello from v2',APP_VERSION=v2
curl -s $URL/; echo
```

EXPECTED

```
✓ Deploying... Done.  revision [bookshelf-api-00005-r1s] ... serving 100 percent of
traffic.
{"books":4,"greeting":"Hello from v2","service":"bookshelf-api","version":"v2"}
```

Step 2 — Split 50/50 with the previous revision and loop

The two revision names are read into variables — nobody types one.

Console path: Cloud Run → bookshelf-api → Revisions → Manage traffic → 50 % each → Save.

CLOUD SHELL

```
NEW=$(gcloud run revisions list --service bookshelf-api --region asia-south1 --
format='value(name)' --limit 1)
OLD=$(gcloud run revisions list --service bookshelf-api --region asia-south1 --
format='value(name)' --limit 2 | tail -n1)
gcloud run services update-traffic bookshelf-api --region asia-south1 --to-revisions
$NEW=50,$OLD=50
for i in $(seq 6); do curl -s $URL/ | grep -o '"version":"v[12]"'; done
```

EXPECTED

```
Traffic:
 50% bookshelf-api-00004-k2d
 50% bookshelf-api-00005-r1s
"version":"v2"
"version":"v1"
"version":"v2"
"version":"v2"
"version":"v1"
"version":"v1"
```

Per-request weighted routing: 4/2 is as normal as 3/3.

Step 3 — Roll back by pointing traffic, then promote latest again**CLOUD SHELL**

```
gcloud run services update-traffic bookshelf-api --region asia-south1 --to-revisions
$OLD=100; curl -s $URL/; echo
gcloud run services update-traffic bookshelf-api --region asia-south1 --to-latest
```

EXPECTED

```
{"books":4,"greeting":"Bookshelf catalogue","service":"bookshelf-api","version":"v1"}
# back to v1 in seconds, no build
```

Step 4 — Read the logs — and find the shutdown line from a scaled-away instance

Console path: Cloud Run → bookshelf-api → Logs → filter box: SIGTERM.

CLOUD SHELL

```
gcloud run services logs read bookshelf-api --region asia-south1 --limit 20
gcloud run services logs read bookshelf-api --region asia-south1 --log-
filter='textPayload:SIGTERM' --limit 5
```

EXPECTED

```
2026-09-07 10:52:07 GET 200 61ms /
2026-09-07 10:38:10 GET 504 30021ms /slow?seconds=45
2026-09-07 10:52:07 bookshelf-api: SIGTERM received, in-flight requests finished,
exiting cleanly
```

Step 5 — Read the revision history

Console path: Cloud Run → bookshelf-api → Revisions → click one → YAML / Variables & secrets.

CLOUD SHELL

```
gcloud run revisions list --service bookshelf-api --region asia-south1
gcloud run revisions describe $NEW --region asia-south1 --
format='yaml(spec.containers[0].env,spec.timeoutSeconds)'
```

EXPECTED

REVISION	ACTIVE	SERVICE	DEPLOYED	DEPLOYED
BY				
✓ bookshelf-api-00005-r1s	yes	bookshelf-api	2026-09-07 10:49:33 UTC	
you@example.com				
✓ bookshelf-api-00004-k2d		bookshelf-api	2026-09-07 10:44:10 UTC	
you@example.com				
...				
spec:				
containers:				
- env:				
- name: GREETING				
value: Hello from v2				
timeoutSeconds: 120				

CHECK YOURSELF

- Five or more revisions from one image digest — every change today was configuration.
- The loop mixed v1 and v2; the rollback answered v1 in seconds.
- The SIGTERM line is in the logs.

Troubleshooting

Symptom	Cause	Fix
--set-env-vars wiped GREETING	set replaces the whole set	Use --update-env-vars
update-traffic: revision not found	A name was typed by hand	Use \$NEW / \$OLD from step 2
No SIGTERM line	No instance has been scaled away yet	Wait a few minutes after the traffic moves, or check after teardown in Logs Explorer
Revisions tab shows fewer than expected	You are looking at another service	Select bookshelf-api

Summary

Configuration, routing and observation are the three things you do to a service every week after the first deploy — and none of them needs a rebuild. The revision list is the change history; the log line plus the revision name tell you what broke and which deploy did it. Keep the service running — Lab 4 adds a second one in front of it.

Lab 4 — Two services talking — web calls api with its own identity

Estimated time	Difficulty	Environment	Kit
20 min	...	Cloud Shell, region asia-south1	gcp-cloudrun-labs-all.zip (bookshelf-api/ + labs.sh + teardown.sh)

OBJECTIVE

The supplied bookshelf-web service deployed in front of bookshelf-api, each running as its own service account, the api requiring authentication and accepting exactly one caller — proven by removing the binding and putting it back. Then the whole session torn down.

PREREQUISITES

- Labs 2–3 completed: bookshelf-api is running; \$AR is set (re-run the first line of Lab 2 step 1 if the shell is new).
- You are inside gcp-cloudrun-labs/bookshelf-api (the commands step into ../bookshelf-web where needed).

bookshelf-web is supplied complete. Read app.py once: before every call to the api it asks the metadata server for an ID token whose audience is the api's URL and sends it as a Bearer header. No key, no secret, nothing to edit.

Steps

Step 1 — Build the web image with Cloud Build

Console path: Cloud Build → History shows the build; Artifact Registry → bookshelf now lists two images.

CLOUD SHELL

```
PROJECT=$(gcloud config get-value project); R=asia-south1
AR=asia-south1-docker.pkg.dev/$PROJECT/bookshelf
cat ../bookshelf-web/app.py | head -40
(cd ../bookshelf-web && gcloud builds submit --tag $AR/bookshelf-web:1.0)
gcloud artifacts docker images list $AR --include-tags
```

EXPECTED

```
... PUSH ... 1.0: digest: sha256:2c8e4a1f... size: 2202 STATUS: SUCCESS
IMAGE DIGEST TAGS
CREATE_TIME
asia-south1-docker.pkg.dev/.../bookshelf/bookshelf-api sha256:9e1d5c7a3b4f... 1.0 ...
asia-south1-docker.pkg.dev/.../bookshelf/bookshelf-web sha256:2c8e4a1f7d9b... 1.0 ...
```

Step 2 — One identity per service — with no roles at all

Console path: IAM & Admin → Service accounts → Create (twice): bookshelf-api-sa, bookshelf-web-sa. Grant no roles.

CLOUD SHELL

```
gcloud iam service-accounts create bookshelf-api-sa --display-name="bookshelf-api runtime"
gcloud iam service-accounts create bookshelf-web-sa --display-name="bookshelf-web runtime"
```

EXPECTED

```
Created service account [bookshelf-api-sa].
Created service account [bookshelf-web-sa].
```

Step 3 — Make the api require authentication and run as its own identity

A configuration change, so a new revision. From now on an anonymous call gets 403.

Console path: Cloud Run → bookshelf-api → Edit & deploy new revision → Security: service account bookshelf-api-sa, Require authentication → Deploy.

CLOUD SHELL

```
gcloud run services update bookshelf-api --region $R --service-account bookshelf-api-sa@$PROJECT.iam.gserviceaccount.com --no-allow-unauthenticated
API_URL=$(gcloud run services describe bookshelf-api --region $R --
format='value(status.url)')
curl -s -o /dev/null -w "anonymous: HTTP %{http_code}\n" $API_URL/
```

EXPECTED

```
✓ Deploying... Done. revision [bookshelf-api-00006-w3q] ... serving 100 percent of
traffic.
anonymous: HTTP 403
```

Step 4 — Let ONLY web's identity invoke the api

Console path: Cloud Run → bookshelf-api → Permissions → Add principal → bookshelf-web-sa@... → Cloud Run Invoker → Save.

CLOUD SHELL

```
gcloud run services add-iam-policy-binding bookshelf-api --region $R \
--member=serviceAccount:bookshelf-web-sa@$PROJECT.iam.gserviceaccount.com --
role=roles/run.invoker
```

EXPECTED

```
Updated IAM policy for service [bookshelf-api].
bindings:
- members:
  - serviceAccount:bookshelf-web-sa@sandbox-rohan-07.iam.gserviceaccount.com
  role: roles/run.invoker
```

Step 5 — Deploy web — public, as web-sa, pointing at the api — and open it

Console path: Cloud Run → Deploy container → image bookshelf-web:1.0 → Allow unauthenticated → Security: bookshelf-web-sa → Variables: API_URL = the api's URL → Create.

CLOUD SHELL

```
gcloud run deploy bookshelf-web --image $AR/bookshelf-web:1.0 --region $R \
--service-account bookshelf-web-sa@$PROJECT.iam.gserviceaccount.com \
--set-env-vars API_URL=$API_URL --allow-unauthenticated
WEB_URL=$(gcloud run services describe bookshelf-web --region $R --
format='value(status.url)')
echo "open: $WEB_URL"; curl -s $WEB_URL/ | grep -o "<li>.*</li>" | head -4
```

EXPECTED

```
Service URL: https://bookshelf-web-482915730261.asia-south1.run.app
<li><b>The Pragmatic Programmer</b> - Hunt & Thomas (1999)</li>
<li><b>Designing Data-Intensive Applications</b> - Kleppmann (2017)</li>
<li><b>Site Reliability Engineering</b> - Beyer et al. (2016)</li>
<li><b>Release It!</b> - Nygard (2018)</li>
```

Open the URL in a browser: the page names the api URL and version and lists four books — web reached an api that refuses everyone else.

CHECK YOURSELF

- Anonymous curl to the api: 403.
- The page lists four books: web's identity got through.
- `gcloud run services get-iam-policy bookshelf-api --region $R` shows exactly one invoker.

Step 6 — Prove the binding is what matters — remove it, reload, restore it

Tokens are cached briefly, so give it ~20 seconds.

Console path: Cloud Run → bookshelf-api → Permissions → remove the web-sa row → Save; reload the page; add it back.

CLOUD SHELL

```
gcloud run services remove-iam-policy-binding bookshelf-api --region $R \
  --member=serviceAccount:bookshelf-web-sa@$PROJECT.iam.gserviceaccount.com --
  role=roles/run.invoker
sleep 20; curl -s $WEB_URL/ | grep -o "could not reach[^<]*"
gcloud run services add-iam-policy-binding bookshelf-api --region $R \
  --member=serviceAccount:bookshelf-web-sa@$PROJECT.iam.gserviceaccount.com --
  role=roles/run.invoker
sleep 20; curl -s $WEB_URL/ | grep -o "<li>.*</li>" | head -1
```

EXPECTED

```
could not reach bookshelf-api: 403 Client Error: Forbidden for url: https://bookshelf-
api-....run.app/
<li><b>The Pragmatic Programmer</b> - Hunt & Thomas (1999)</li>
```

Step 7 — Teardown — both services, both identities, the repository, local images

Console path: Cloud Run → select both services → Delete. IAM & Admin → Service accounts → delete both. Artifact Registry → bookshelf → Delete.

CLOUD SHELL

```
cd .. && bash teardown.sh
gcloud run services list --region $R; gcloud artifacts repositories list --
location=$R; docker images
```

EXPECTED

```
Cloud Run session resources removed.
Listed 0 items.
Listed 0 items.
REPOSITORY    TAG      IMAGE ID      CREATED      SIZE
```

Troubleshooting

Symptom	Cause	Fix
Page shows 403 with the binding in place	Service account propagation (~30 s), or the binding is on the wrong service	Wait, reload; get-iam-policy bookshelf-api
Page shows connection error	API_URL not set on web	Re-run step 5 with --set-env-vars API_URL=\$API_URL
deploy: service account does not exist	IAM propagation delay	Wait 30 s and re-run
Anonymous curl still 200	Step 3's update did not apply	gcloud run services describe bookshelf-api ... grep -i auth; re-run step 3

Challenge (fast finishers) — take the api off the internet entirely

Add --ingress internal to the api and route web's egress through the VPC so its calls arrive from inside: gcloud compute networks subnets update default --region=\$R --enable-private-ip-google-access, then gcloud run services update bookshelf-api --region \$R --ingress internal, then gcloud run services update bookshelf-web --region \$R --network default --subnet default --vpc-egress all-traffic. Anonymous curl now gets 404 (not 403) —

the api is invisible from the internet — and the page still works. This is the pattern from the landscape session's networking chapter.

Summary

Two services, one identity each, one invoker binding between them, and no secret anywhere. The page worked or failed on that binding alone — which is the whole of service-to-service trust on Cloud Run. The challenge adds the network layer on top: internal ingress plus VPC egress makes the api unreachable from outside, not merely unauthorised.

Appendix — Environment, versions and the kit

Item	Pinned	Note
Cloud Shell	gcloud 583.x, Docker 28.x, psql, kubectl pre-installed	browser only; nothing to install
Region	asia-south1	all labs
Base images	python:3.13-slim (runtime and builder); python:3.13 for the naive comparison	pinned major.minor
Flask / gunicorn	3.1.3 / 26.2.0	pinned in requirements.txt
Artifact Registry	asia-south1-docker.pkg.dev/<project>/bookshelf	Docker, standard mode
Cloud Run facts (verified Sept 2026)	PORT injected (8080); listen within 4 min; SIGTERM → 10 s → SIGKILL; timeout default 5 min, max 60; concurrency 80 default / 1000 max; up to 8 vCPU / 32 GiB	container runtime contract + quotas pages

The kit

- bookshelf-api/ — app.py, gunicorn.conf.py, requirements.txt, Dockerfile.naive, Dockerfile, .dockerignore — supplied complete, never edited
- bookshelf-web/ — app.py, requirements.txt, Dockerfile — the second service for Lab 4, supplied complete
- labs.sh — the four labs as one command sheet (paste block by block)
- teardown.sh — both services, both service accounts, repository, local images

What the supplied files do

File	Role in the labs
app.py	the catalogue API: /, /books, /books/<id>, /healthz (health contract), /whoami (instance + uptime — makes scaling and cold starts visible), /slow?seconds=N (timeouts)
gunicorn.conf.py	the process side of the contract: bind 0.0.0.0:\$PORT, 1 worker × 8 threads, graceful_timeout 8 s, logs to stdout, on_exit shutdown line
Dockerfile.naive	the first Dockerfile everyone writes: full base, COPY everything, unpinned pip, root, shell-form CMD
Dockerfile	the hardened one: multi-stage, slim, pinned, non-root, exec-form CMD
bookshelf-web/app.py	renders the catalogue fetched from the api; mints an ID token for the api URL with its own identity (google-auth + metadata server) — Lab 4

Volatile facts (versions, limits, console navigation) were verified against docs.cloud.google.com on 7 September 2026. Re-verify before each delivery.